

HDFC Custom LLM Development Pipeline — Complete Project Understanding & Team Division

1. Project in One Sentence

A controlled platform that helps HDFC teams **build, test, approve, deploy, integrate, and monitor banking-specific LLMs safely and repeatedly.**

The core flow is:

```
graph TD; A[Banking Data] --> B[Data Preparation]; B --> C[Approved Dataset]; C --> D[Fine-tuning / LoRA / QLoRA]; D --> E[Evaluation]; E --> F[Approval]; F --> G[Model Registry]; G --> H[Private Deployment]; H --> I[Model Gateway]; I --> J[Banking Applications]; J --> K[Monitoring + Feedback];
```

Banking Data
↓
Data Preparation
↓
Approved Dataset
↓
Fine-tuning / LoRA / QLoRA
↓
Evaluation
↓
Approval
↓
Model Registry
↓
Private Deployment
↓
Model Gateway
↓
Banking Applications
↓
Monitoring + Feedback

2. What Problem Is HDFC Trying to Solve?

The platform prevents different teams from independently creating models with different datasets, evaluation methods, deployment processes, and security controls.

Without a common pipeline:

Team A → Model A → Different Dataset → Different Evaluation

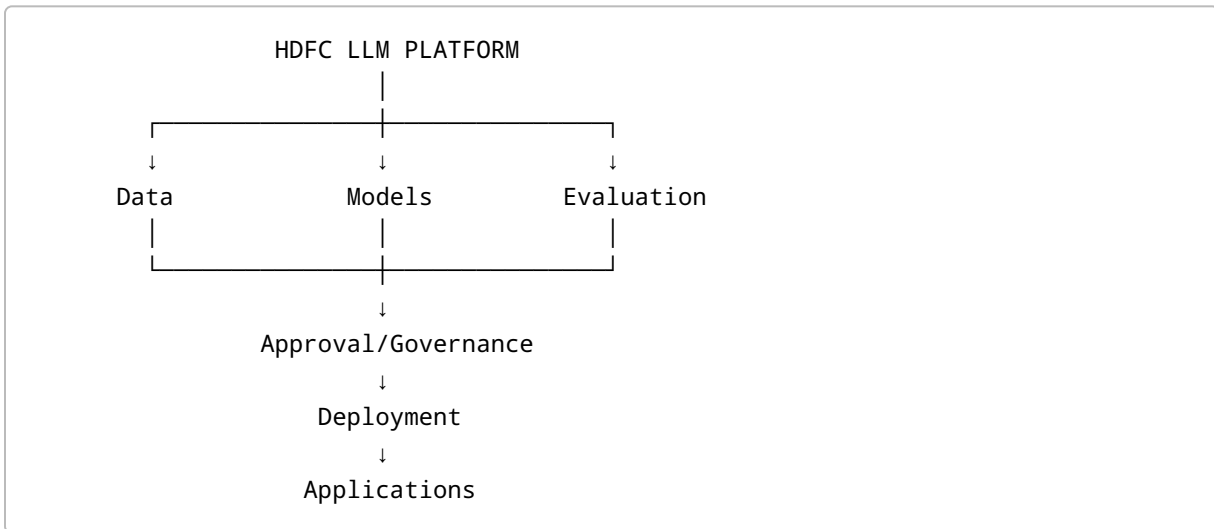
Team B → Model B → Different Dataset → Different Evaluation

Team C → Model C → Different Deployment → Different Security

This can cause:

- Inconsistent datasets
- Inconsistent model quality
- Difficult model comparison
- Duplicated work
- Security risks
- Model-risk problems
- No clear ownership
- Difficult rollback

The platform creates a common process:



3. What Are We Actually Building?

The project has two major parts.

Part A — LLM/ML Pipeline

Mainly handled by the Data Science team:

```
Data
↓
Clean
↓
De-identify
↓
Create Dataset
↓
Fine-tune
↓
Evaluate
↓
Register Model
↓
Deploy
```

Part B — Control/Product Interface

The Web Developer builds a web application through which authorized users can manage the pipeline.

Example:

HDFC Custom LLM Platform	
Dashboard	Dataset Status ██████████ Approved
Datasets	
Fine-tuning	Training Jobs ██████████ Running
Evaluation	Model Evaluation Accuracy: 92%
Models	Safety: 97%
Deployment	Deployment • Production
Monitoring	

4. Dataset Registration

The platform first records information about the banking data.

For example:

```
Dataset:
Home Loan Procedures

Owner:
Banking Operations

Purpose:
LLM Training

Classification:
Internal

Version:
1.0

Status:
Pending Approval
```

The system records:

- Data owner
- Data source
- Purpose
- Classification
- Access permissions
- Retention
- Geography
- Effective date

This is part of data governance and lineage.

5. Data Scanning

Before data enters the training pipeline, it must be checked.

The system can check for:

```
Passwords
Payment credentials
Personal information
Authentication information
Secrets
Unsupported files
Malware
```

If something dangerous is detected:

```
Data
↓
Security Scan
↓
❌ Problem
↓
Quarantine / Review
```

This is especially important for a banking project.

6. Data Preparation

After scanning:

```
Raw Data
↓
Parse
↓
Normalize
↓
De-identify
↓
Create Records
↓
Remove Duplicates
↓
Split Dataset
↓
Quality Checks
↓
Freeze Version
```

Example:

```
Customer: Ankush  
Account: 123456789  
Phone: 98XXXXXXX
```

Could be transformed into:

```
Customer: <CUSTOMER>  
Account: <ACCOUNT_ID>  
Phone: <PHONE>
```

The exact implementation depends on the approved project approach.

7. Dataset Creation

The project uses structured task records rather than random text.

Example:

```
{  
  "instruction": "Explain home loan eligibility",  
  "context": "Approved banking procedure...",  
  "response": "...",  
  "label": "home_loan",  
  "citation": "...",  
  "refusal": false,  
  "escalation": false  
}
```

The dataset should be:

- Task-specific
 - Structured
 - Versioned
 - Traceable
 - Quality checked
-

8. Dataset Splitting

The dataset needs controlled partitions:

```
Training
Validation
Testing
Holdout
```

The project also considers:

- Contamination checks
- Entity separation
- Time separation
- Immutable partitions

The goal is to prevent artificially high evaluation results.

9. Fine-Tuning

The project uses model adaptation techniques such as LoRA and QLoRA.

Conceptually:

```
Approved Base Model
      +
Approved Dataset
      +
LoRA / QLoRA
      ↓
Fine-tuned Adapter
```

Instead of modifying the entire large model, LoRA trains a smaller adapter.

```
Large Base Model
  |
  | Frozen
  ↓
LoRA Adapter
  |
```

↓
Banking-specific behavior

10. Training Run

An ML engineer selects:

Base Model
Dataset Version
LoRA Configuration
Compute Configuration
Checkpoint Policy
Evaluation Plan

The system records:

Code Version
Libraries
Container
Seed
Hardware
Hyperparameters
Dataset Hash
Metrics
Checkpoints
Resource Usage

This provides reproducibility.

11. Experiment Tracking

The project recommends experiment tracking such as MLflow.

Example:

Experiment: HDFC-Banking-V1

Learning Rate: 0.0002
Epochs: 3

Batch Size: 16

Training Loss: 0.82

Validation Loss: 0.91

GPU Usage: ...

Training Time: ...

This makes it possible to compare different experiments.

12. Evaluation

The model must be systematically evaluated.

Evaluation can include:

Banking terminology
Intent understanding
Extraction
Summarization
Structured generation
Grounded answers
Refusal
Escalation
Citations
Safety
Privacy
Security
Bias
Robustness
Multilingual behavior
Latency

The important point is:

We do not simply decide that a model is good because it produces nice answers.

13. Critical Failure vs Average Score

Suppose:

Model A
Accuracy = 95%

Model B
Accuracy = 92%

Model A looks better.

But suppose:

Model A:
Invents banking eligibility rules ❌

Model B:
Doesn't invent eligibility rules ✅

Model A should not automatically be approved.

The project requires risk-tiered evaluation and critical-failure gates.

14. Safety Evaluation

The model can be tested for:

Prompt injection
Sensitive-data leakage
Unsupported advice
Policy conflicts
Social engineering
Coercion
Discrimination
Unsafe certainty
Missing escalation
Malicious instructions

This is an important responsibility for the Data Science team.

15. Model Registry

After evaluation, the model is stored in a Model Registry.

Think of the registry as:

The official warehouse of approved model artifacts and their evidence.

It can contain:

```
Model ID
Version
Base Model
Adapter
Dataset
Training Information
Evaluation Results
Model Card
Limitations
Owner
Approvals
Signature
Expiry
Deployment History
Incidents
```

The project recommends MLflow Model Registry or an approved internal registry.

16. Approval Gate

A model does not automatically go to production.

The process is:

```
Model
↓
Evaluation
↓
Safety
↓
Privacy
↓
```

```
Security
↓
Performance
↓
Human Review where required
↓
APPROVED
```

If a critical gate fails:

✗ NOT APPROVED

17. Deployment

Only an approved and signed artifact should be deployed.

Conceptually:

```
Model Registry
↓
Signature Verification
↓
Approval Verification
↓
Private Endpoint
↓
Canary
↓
Monitoring
↓
Production
```

Potential serving technologies include:

- vLLM
- Hugging Face Text Generation Inference
- Kubernetes

18. Canary Deployment

Instead of sending all traffic to a new model immediately:

```
New Model
↓
5% Traffic
↓
Monitor
↓
Good?
↓
25%
↓
50%
↓
100%
```

If there is a problem:

```
New Model ✗
↓
Rollback
↓
Previous Approved Model
```

19. Model Gateway

Applications should not directly access model artifacts.

Instead:

```
Banking Application
↓
API Gateway
↓
Model Gateway
↓
Approved Model
```

The gateway can handle:

- Authentication
- Authorization
- Purpose
- Retrieval
- Prompt construction
- Input validation
- Output validation
- Guardrails
- Tracing
- Fallback
- Rate limits

This is one reason the project is much broader than a chatbot.

20. Retrieval

Dynamic regulatory and policy information should remain outside the model weights.

For example, if a banking policy changes:

```
Old Policy
  ↓
New Policy
```

We should not necessarily retrain the entire model.

Instead:

```
Authoritative Policy
  ↓
Governed Retrieval
  ↓
Current Context
  ↓
Model
```

The model receives the current approved information.

21. Prompting

The system prompt can define:

```
Banking role
Permitted purpose
Authoritative sources
Prohibited actions
Escalation rules
Output format
Uncertainty behavior
```

A structured response can look like:

```
{
  "answer": "...",
  "citations": [],
  "confidence": "...",
  "assumptions": [],
  "missing_information": [],
  "policy_flags": [],
  "escalation_required": false
}
```

22. Agentic Behavior

The project allows controlled agentic behavior, but with restrictions.

The model should not directly execute sensitive banking actions.

Instead:

```
LLM
↓
Authorized Tool
↓
Validation
↓
Banking System
↓
```

Verified Result

↓

LLM

The model must not claim that an action succeeded unless a verified tool result confirms it.

Autonomous transaction decisions/customer commitments are not authorized for the initial model scope.

23. Monitoring

After deployment, monitoring continues.

Monitor:

Availability
Latency
Token Usage
Errors
Guardrail Events
Fallback
Refusals
Quality
Drift
Endpoint Saturation
Incidents

The lifecycle becomes:

Deploy
↓
Monitor
↓
Detect Problem
↓
Investigate
↓
Rollback / Fix
↓
Evaluate Again

24. Frontend Requirements

The frontend is not primarily a chatbot UI.

The project defines product surfaces such as:

Portfolio Overview

Displays:

- Approved Models
- Dataset Status
- Active Runs
- Evaluation Status
- Deployed Versions
- Incidents
- Drift
- Approvals

Dataset Workspace

Provides:

- Register Source
- Classification
- Transformation
- De-identification Results
- Quality Report
- Version Comparison
- Approval

Run Builder

Allows the ML engineer to select:

- Base Model
- Dataset
- LoRA Parameters
- Compute
- Checkpoint Policy
- Evaluation Plan

Then:

[Validate]
[Start Training]

Run Monitor

Displays:

Training Status
Metrics
GPU / Resource Usage
Checkpoints
Logs
Failures
Cancel
Resume

Evaluation Center

Displays:

Evaluation Suites
Baseline Comparison
Failure Clusters
Severity
Human Review
Red-team Findings
Decision

Model Registry

Displays:

Model
Version
Artifact
Lineage
Model Card
Limitations
Approvals

Expiry
Deployments
Incidents

Deployment Console

Displays:

Endpoint
Capacity
Canary Percentage
SLO
Guardrails
Current Version
Previous Version

[Deploy]

[Rollback]

Integration Explorer

Allows developers to test:

Request
↓
Retrieval
↓
Prompt
↓
Model
↓
Structured Response
↓
Trace
↓
Latency
↓
Guardrail Result

This is a model testing/integration interface, not the primary product.

25. Recommended Technology Stack

Layer	Technology
Frontend	Next.js, React, Tailwind CSS
Backend/API	FastAPI, Pydantic
Database	PostgreSQL
Object Storage	S3-compatible storage
Data Format	Apache Parquet / Delta Lake
Fine-tuning	PyTorch, Transformers, PEFT, TRL
Distributed Compute	DeepSpeed / PyTorch FSDP
Orchestration	Kubernetes
Experiment Tracking	MLflow
Evaluation	lm-evaluation-harness + custom suites + promptfoo/Giskard
Model Registry	MLflow Model Registry/internal registry
Serving	vLLM / TGI
Guardrails	NeMo Guardrails, Presidio, deterministic validation
Monitoring	OpenTelemetry, Prometheus, Grafana, Evidently

26. Team Division — 1 Web Developer + 4 Data Science Members

You — Web Developer / Team Lead

Primary responsibility

Control Plane + Frontend + API Integration

Technologies:

```
Next.js
React
JavaScript
Tailwind CSS
```

API Integration
Frontend State Management

You build:

Dashboard
Dataset UI
Training UI
Evaluation UI
Model Registry UI
Deployment UI
Integration Explorer

You also coordinate API contracts with the backend/MLOps member.

27. DS Member 1 — Data Engineering / Dataset

Ownership

Raw Banking Data
↓
Data Preparation
↓
Approved Dataset

Responsibilities:

- Data collection
- Data registration
- Data classification
- Parsing
- Normalization
- De-identification
- Deduplication
- Labeling
- Dataset splitting
- Quality checks
- Dataset versioning
- Lineage
- Dataset cards

Main areas:

```
services/  
└─ data-preparation/  
└─ dataset-registry/
```

28. DS Member 2 — Fine-tuning / ML Engineering

Ownership

```
Approved Dataset  
  +  
Approved Base Model  
  ↓  
LoRA / QLoRA  
  ↓  
Model Adapter
```

Responsibilities:

- Base-model selection
- LoRA
- QLoRA
- Training configuration
- GPU jobs
- Checkpoints
- Training metrics
- Experiment tracking
- Reproducibility

Main areas:

```
services/  
└─ run-orchestrator/  
  
packages/  
└─ finetuning/
```

29. DS Member 3 — Evaluation + Model Risk

Ownership

```
Candidate Model
  ↓
Evaluation
  ↓
Safety
  ↓
Privacy
  ↓
Security
  ↓
Approval Evidence
```

Responsibilities:

- Banking evaluation
- Quality metrics
- Groundedness
- Hallucination testing
- Safety
- Privacy
- Bias
- Robustness
- Prompt injection
- Red-team testing
- Human evaluation
- Regression tests

Main areas:

```
services/
└─ evaluation-service/

packages/
└─ evaluations/
```

30. DS Member 4 — Backend + MLOps

Ownership

```
graph TD;
    A[Approved Model] --> B[Model Registry];
    B --> C[Deployment];
    C --> D[Private Endpoint];
    D --> E[Monitoring];
    E --> F[Rollback];
```

Responsibilities:

- FastAPI
- Pydantic
- PostgreSQL integration
- Model Registry
- vLLM/TGI
- Model Gateway
- Kubernetes
- Deployment
- Monitoring
- Logging
- Tracing
- Rollback
- SLOs
- Model lifecycle

Main areas:

```
services/
├─ model-registry-adapter/
├─ model-gateway/
└─ monitoring-service/
```


31. Complete Repository Architecture

```
hdfc-custom-llm-development-pipeline/
```

```
|
|— apps/
|   |
|   |— control-plane/
|   |   |
|   |   |— app/
|   |   |— components/
|   |   |— features/
|   |   |   |
|   |   |   |— dashboard/
|   |   |   |— datasets/
|   |   |   |— runs/
|   |   |   |— evaluations/
|   |   |   |— models/
|   |   |   |— deployments/
|   |   |   |— integrations/
|   |   |— services/
|   |   |— lib/
|   |— review-portal/
|
|— services/
|   |
|   |— dataset-registry/
|   |— data-preparation/
|   |— run-orchestrator/
|   |— evaluation-service/
|   |— model-registry-adapter/
|   |— model-gateway/
|   |— monitoring-service/
|
|— packages/
|   |
|   |— contracts/
|   |— banking-taxonomy/
|   |— data-quality/
|   |— finetuning/
|   |— evaluations/
|   |— guardrails/
|
|— configs/
|   |
|   |— observability/
|   |— base-models/
|   |— lora/
```

```
|
|
|— data/
|   |— schemas/
|   |— synthetic/
|   └─ evaluation-fixtures/
|
|— infrastructure/
|   |— containers/
|   |— kubernetes/
|   └─ migrations/
|
|— tests/
|   |— unit/
|   |— integration/
|   └─ end-to-end/
|
|— docs/
|   |— architecture/
|   |— data-governance/
|   |— model-risk/
|   |— api/
|   └─ runbooks/
|
|— .env.example
|— docker-compose.yml
|— README.md
└─ LICENSE
```

32. Folder Ownership

Folder	Owner
apps/control-plane	You
apps/review-portal	You + DS3
dataset-registry	DS1 + DS4
data-preparation	DS1
run-orchestrator	DS2 + DS4
evaluation-service	DS3
model-registry-adapter	DS4

Folder	Owner
model-gateway	DS4 + You
monitoring-service	DS4
contracts	You + DS4
banking-taxonomy	DS1 + DS3
data-quality	DS1
finetuning	DS2
evaluations	DS3
guardrails	DS3 + DS4
infrastructure	DS4
tests	Everyone
docs	Everyone; you coordinate

33. Development Order

Do not build the entire frontend first.

Use this order:

```

PHASE 1
Requirements + Architecture
    ↓
PHASE 2
API Contracts
    ↓
PHASE 3
Dataset Pipeline
    ↓
PHASE 4
Fine-tuning Pipeline
    ↓
PHASE 5
Evaluation
    ↓
PHASE 6
Model Registry

```

```
↓
PHASE 7
Deployment
↓
PHASE 8
Frontend Integration
↓
PHASE 9
Monitoring
↓
PHASE 10
Testing + Demo
```

However, frontend and ML development should happen in parallel where possible.

34. What You Should Do First as Team Lead

Your first task should not be:

"Create the HDFC chatbot."

Instead:

Day 1

Create the GitHub repository and finalize:

```
Frontend
↓
FastAPI
↓
Dataset Services
↓
Training
↓
Evaluation
↓
Registry
↓
Deployment
```

Day 2

Define API contracts such as:

```
POST /v1/datasets
POST /v1/datasets/{id}/prepare
POST /v1/datasets/{id}/approve

POST /v1/runs
GET /v1/runs/{id}

POST /v1/evaluations

POST /v1/models/register
POST /v1/models/{id}/promote

POST /v1/inference

POST /v1/feedback

POST /v1/deployments/{id}/rollback
```

Then begin the Next.js frontend with mock data while the DS team builds the actual ML pipeline.

35. First End-to-End MVP

Do not attempt to implement the entire enterprise platform at once.

Build this first:

```
1. Register Synthetic Banking Dataset
    ↓
2. Prepare Dataset
    ↓
3. Create Training Run
    ↓
4. Fine-tune / Adapt Small Approved Model
    ↓
5. Evaluate
    ↓
6. Register Model
    ↓
7. Approve
```

- ↓
8. Deploy Local/Private Endpoint
 - ↓
 9. Call Through Model Gateway
 - ↓
 10. Show Result in Integration Explorer

Your frontend should display every stage.

This gives your team a complete end-to-end demonstration of the actual Custom LLM Development Pipeline.

36. Most Important Distinction

The project should be understood as:

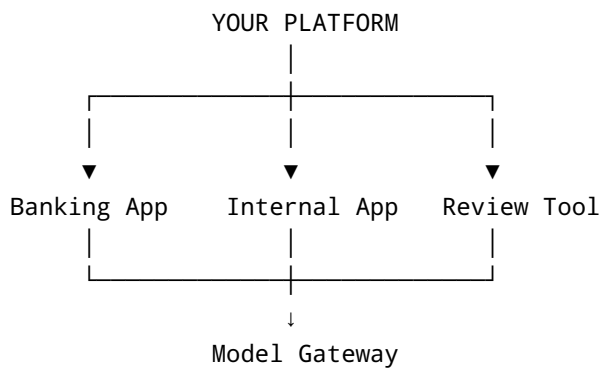
✗ NOT:

"Let's build an HDFC chatbot."

✓ ACTUALLY:

"Let's build a governed platform that allows HDFC teams to create, evaluate, approve, deploy, integrate, and monitor banking-specific LLM models."

A chatbot or banking application can eventually sit on top of the platform:



↓
Approved LLM